

Data Structures & Algorithms — Course Notes

CS Core Course, Semester 5

Topic 1: Arrays

An array is a fixed-size, contiguous block of memory that stores elements of the same data type. Because elements are stored contiguously, any element can be accessed directly using its index in constant time, $O(1)$. Insertion or deletion in the middle of an array is expensive, $O(n)$, because all subsequent elements must be shifted.

Key operations: access $O(1)$, search $O(n)$, insertion/deletion $O(n)$.

Use when: the number of elements is known in advance and random access is frequent.

Topic 2: Linked Lists

A linked list is a linear data structure where each element (node) holds its data plus a pointer to the next node. Unlike arrays, linked lists do not require contiguous memory, so insertion and deletion at a known position are $O(1)$ once you have a reference to that node. The trade-off is that random access degrades to $O(n)$ since you must traverse from the head.

Variants: singly linked list, doubly linked list, circular linked list.

Key operations: access/search $O(n)$, insertion/deletion at head $O(1)$.

Use when: frequent insertions/deletions are needed and random access is rare.

Topic 3: Stacks

A stack is a Last-In-First-Out (LIFO) structure supporting two primary operations: push (add to top) and pop (remove from top), both $O(1)$. Stacks are commonly implemented using arrays or linked lists.

Real-world uses: function call stacks, undo mechanisms in editors, expression evaluation (infix to postfix conversion), and backtracking algorithms.

Topic 4: Queues

A queue is a First-In-First-Out (FIFO) structure supporting enqueue (add to rear) and dequeue (remove from front), both $O(1)$ with a proper implementation (e.g., circular buffer or linked list). Variants include circular queues, priority queues, and double-ended queues (dequeues).

Real-world uses: task scheduling, breadth-first search traversal, print job management, and buffering in streaming systems.

Topic 5: Trees

A tree is a hierarchical data structure consisting of nodes connected by edges, with one root node and no cycles. A binary tree restricts each node to at most two children. A Binary Search Tree (BST) additionally

maintains the invariant that left children are smaller and right children are larger than their parent, enabling $O(\log n)$ average-case search, insertion, and deletion when balanced.

Traversals: in-order, pre-order, post-order, and level-order (BFS).

Balanced variants: AVL trees and Red-Black trees guarantee $O(\log n)$ operations even in the worst case by enforcing height balance after every insertion/deletion.

Topic 6: Graphs

A graph is a set of vertices (nodes) connected by edges, which may be directed or undirected, and weighted or unweighted. Graphs are typically represented using an adjacency matrix ($O(V^2)$ space, $O(1)$ edge lookup) or an adjacency list ($O(V+E)$ space, better for sparse graphs).

Common algorithms: Breadth-First Search (BFS) for shortest path in unweighted graphs, Depth-First Search (DFS) for connectivity and cycle detection, Dijkstra's algorithm for shortest paths in weighted graphs with non-negative weights, and Kruskal's/Prim's algorithms for minimum spanning trees.

Topic 7: Hashing

A hash table maps keys to values using a hash function that converts a key into an array index, giving average-case $O(1)$ insertion, deletion, and lookup. Collisions (two keys hashing to the same index) are resolved via chaining (linked lists at each bucket) or open addressing (probing for the next free slot).

Trade-off: excellent average-case speed but no ordering guarantee, and worst-case $O(n)$ if the hash function distributes keys poorly.

Topic 8: Sorting Algorithms

Sorting algorithms arrange elements in a defined order. Comparison-based sorts have a theoretical lower bound of $O(n \log n)$.

Bubble/Insertion Sort: $O(n^2)$, simple, good for nearly-sorted or small datasets.

Merge Sort: $O(n \log n)$ guaranteed, stable, needs $O(n)$ extra space; divides recursively and merges sorted halves.

Quick Sort: $O(n \log n)$ average, $O(n^2)$ worst case, in-place, fastest in practice due to cache locality.

Heap Sort: $O(n \log n)$ guaranteed, in-place, not stable; builds a max-heap and extracts the maximum repeatedly.

Topic 9: Searching Algorithms

Linear Search checks each element sequentially and runs in $O(n)$ on unsorted data. Binary Search requires sorted data and repeatedly halves the search space by comparing the target to the middle element, running in $O(\log n)$.

Binary search is the classic example used to teach the divide-and-conquer paradigm, where a problem is split into smaller subproblems solved independently.

Topic 10: Dynamic Programming

Dynamic Programming (DP) is an optimization technique for problems with overlapping subproblems and optimal substructure — meaning the optimal solution can be built from optimal solutions to smaller subproblems. DP avoids redundant computation using either memoization (top-down, caching recursive results) or tabulation (bottom-up, filling a table iteratively).

Classic examples: Fibonacci sequence computation, the 0/1 Knapsack problem, Longest Common Subsequence, and shortest path algorithms like Bellman-Ford.

Topic 11: Greedy Algorithms

A greedy algorithm builds a solution piece by piece, always choosing the option that looks best at the current step, without reconsidering earlier choices. Greedy approaches work only when a problem exhibits the greedy-choice property — a locally optimal choice leads to a globally optimal solution.

Classic examples: Dijkstra's shortest path, Kruskal's/Prim's minimum spanning tree, Huffman coding for data compression, and the activity selection problem.

Topic 12: Time & Space Complexity (Big-O Notation)

Big-O notation describes how an algorithm's runtime or memory usage grows as the input size n grows, focusing on the dominant term and ignoring constants. It expresses an upper bound on growth rate.

Common complexity classes, fastest to slowest growth:

$O(1)$ constant $\rightarrow O(\log n)$ logarithmic $\rightarrow O(n)$ linear $\rightarrow O(n \log n)$ linearithmic $\rightarrow O(n^2)$ quadratic $\rightarrow O(2^n)$ exponential $\rightarrow O(n!)$ factorial.

End of notes. Compiled for course reference and RAG retrieval testing.